

Wallet — Intensified Test Cases

Buyer-side recharge / coin purchase flow | Black-box, user-perspective | Privacy & security focused

Purpose

This document covers wallet test scenarios that a regular manual QA tester typically misses. Every case is executable on a stock Android device with no rooting, no MITM proxy, and no developer-side tooling. The bugs they target are real risks to user privacy, money, and trust: phantom coin credit, double-billing, leaked PII on shared/recorded screens, lost recharges across crashes, and abusable promo flows.

Scope

WalletActivity recharge: PhonePe, Cashfree UPI Intent, Razorpay link, Google Play Billing (IAP), UPI gateway. Coupon & referral offer dialogs that originate inside the wallet screen are in scope. Creator-side withdraw/UPI/Bank/PAN flows are out of scope.

How to use this document

- Run cases in priority order: P0 → P1 → P2. P0 cases are money-impacting or privacy-impacting and must pass every release.
- For each case, record: device + OS version, payment gateway selected, network condition, time-of-day, outcome (pass/fail), and a screen recording for any fail.
- If a case has **Branches**, treat each branch as a separate execution — pass on one branch does not imply pass on the others.
- "Why humans miss it" explains the cognitive blind spot. Read it before executing — it changes how you watch the flow.

Priority legend

P0	Money loss, double-charge, phantom credit, or PII exposure to a third party. Blocker.
P1	Inconsistent state, stuck recharge, refund pathway broken, abusable promo. Must-fix before release.
P2	Visual / UX edge cases, accessibility, minor leakage. Fix in next iteration.

User personas covered

- **First-time recharger** — no SKU history, sees referral offer dialog, may abandon mid-flow.
- **Repeat recharger** — known SKU, knows shortcuts, may double-tap or back-press during payment.
- **Shared-device user** — phone is shared with family, screenshots/notifications/recents leak to next viewer.
- **Low-trust environment** — public Wi-Fi, low battery, battery-saver, weak signal, SIM-less tablet, screen mirror to TV.
- **Accessibility user** — TalkBack on, large font, color-inverted, screen reader announces balances and OTP-like values.
- **Frustrated user** — payment failed once, retries while previous attempt is still settling.

Test Case Bank

Each test case is bordered, color-coded by priority, and self-contained. 'Why humans miss it' explains the blind spot — read before executing.

Section A — Plan Selection & Pricing Integrity

The user has not paid anything yet, but the decisions they make on this screen lock in the amount and the SKU. Bugs here propagate to every downstream gateway.

WAL-A-01 [P0] Coin plan price displayed equals price actually charged (integrity of plan card)	
Objective	Confirm the rupee amount shown on the selected plan card is byte-identical to the amount the gateway charges, including any 2% convenience-fee math.
Preconditions	Logged-in male user, wallet open, at least 3 plans loaded.
Steps	1. Note the exact price text on every plan card. 2. Tap plan #1 and proceed to checkout for each of: PhonePe, Cashfree, Razorpay, GPay, UPI Gateway (one gateway per attempt; cancel before paying). 3. Compare the amount shown in the gateway sheet (and the rupee value in the UPI app intent screen) against the plan card price. 4. Repeat for plan #2 and the largest plan in the list.
Expected	Gateway-side amount == plan card amount for PhonePe, Cashfree, Razorpay, GPay. For UPI Gateway, the gateway-side amount equals plan card price + 2% (rounded). The 2% surcharge is disclosed on the wallet screen <i>before</i> the user taps Pay.
Why humans miss it	UPI Gateway path silently adds a 2% convenience fee in-app — a user who memorises the plan price will be charged more without an in-app disclosure.
Branches	(a) Plan price is a whole number; (b) plan price is a decimal (₹49.50 etc.); (c) plan price is very small (₹1) — verify the 2% rounding does not silently make it ₹2 or ₹0.

WAL-A-02 [P1] Default-selected plan on wallet load is the plan that Pay actually buys

Objective

The first plan in the grid is auto-selected; the 'Add X Coins' button changes text accordingly. Confirm a user who taps the button immediately (without manually tapping a plan card) purchases that exact plan.

Preconditions

Wallet just opened, no plan tapped yet.

Steps

1. Open wallet from Home (cold).
2. Without scrolling or tapping any plan card, tap 'Add Coins'.
3. Observe the gateway sheet amount and the SKU sent to the gateway.

Expected

Amount == price of the first plan card in the visible grid. Coins credited == coin count on first plan card.

Why humans miss it

Default-selection bugs are silent — the button is enabled, the purchase 'works', but the user did not consciously choose the plan.

WAL-A-03 [P1] Rapid plan-card switching before tap-to-pay does not race

Objective

User selects plan A → plan B → plan C within 500 ms, then taps Pay. The purchase must be for plan C, the last-tapped, never a stale earlier one.

Steps

1. Open wallet.
2. Tap plan A, B, C in quick succession (< 1 s total).
3. Tap 'Add Coins'.
4. Confirm the amount on the gateway sheet.

Expected

Gateway amount == plan C price. SKU sent == plan C SKU.

Why humans miss it

Listener / observer ordering bugs surface only under fast input. QA usually waits between taps.

WAL-A-04 [P0] Banner-image / promotional artwork cannot be tapped to bypass plan selection

Objective

Confirm the dynamically-loaded banner image at the top of the wallet is purely decorative — it does not navigate to a different store, deep-link, or trigger a purchase without user consent.

Steps

1. Tap the banner image once.
2. Long-press the banner image.
3. Double-tap the banner image rapidly.

Expected

No navigation, no payment, no deep-link, no toast. Image is non-interactive.

Why humans miss it

Banner is loaded from a backend-controlled URL — if it ever becomes tappable, a compromised CDN response could redirect users to a phishing site.

WAL-A-05 [P2] Plan grid renders correctly with 1, 2, 4, 7, and 13 plans

Objective

Visual integrity across uncommon plan counts. The grid is a 3-column layout — counts that are not multiples of 3 stress the last row.

Steps

1. Coordinate with backend to return plan lists of size 1, 2, 4, 7, 13 (or test against each environment that has these counts).
2. Confirm no plan is clipped, no blank card is tappable, scroll still works.

Expected

Last row never contains a phantom tappable empty card. No crash if size == 0.

Why humans miss it

GridLayoutManager with 3 columns + empty / off-by-one item counts is a classic source of crashes on production data shapes nobody tested on staging.

WAL-A-06 [P1] 'Save X%' label on plan card matches the actual savings vs the cheapest per-coin plan

Objective

The save-percent badge is computed by the backend. Sanity-check that the math is consistent — a wrong badge is a misleading-advertising risk.

Steps

1. For every plan, compute (price / coins). Find the cheapest per-coin plan; treat that as baseline.
2. For every other plan, compute the expected save %.
3. Compare against the badge on the card.

Expected

Computed save % matches the badge within ± 1 percentage point.

Why humans miss it

ASCI / consumer-protection complaints have killed promo campaigns at competing apps. The math is server-driven but shown by the app, so the app is the visible offender.

Section B — Payment Gateway Assignment

The user's assigned gateway is fetched from the login response on every onResume. The wallet branches into 5 gateway code paths. Verify the right path is chosen, deterministically.

WAL-B-01 [P0] Payment gateway assignment is consistent across the same user session

Objective

Gateway type is fetched on Wallet open AND on every onResume (via a login call). Confirm a user who opens wallet, backgrounds the app, and resumes does not get switched to a different gateway mid-flow.

Preconditions

User account with a stable assigned gateway.

Steps

1. Open wallet. Note which gateway sheet appears when you tap 'Add Coins' (cancel before paying).
2. Press Home. Wait 30 seconds.
3. Bring app to foreground. Tap 'Add Coins' again.
4. Note the gateway again. Repeat for 5 background-resume cycles.

Expected

Same gateway every time within a session.

Why humans miss it

The wallet calls the login API on every onResume to refresh the gateway. If the backend flips the user between gateways, the user experiences inconsistent behavior with no explanation.

WAL-B-02 [P0] If gateway field is empty / null / unknown, no payment flow starts

Objective

Verify graceful failure when paymentGateway is empty, contains an unexpected value, or is mis-cased ('PhonePe' vs 'phonepe').

Steps

1. Without changing anything in the app, log out and back in during a known backend deploy window.
2. Repeatedly open wallet and tap 'Add Coins'. Look for the case where the toast 'Invalid Payment Gateway' appears.
3. Try tapping Pay rapidly several times when toast appears.

Expected

Toast appears; no payment starts; no coins credited; no loading spinner gets stuck.

Why humans miss it

Switch-statement on gateway string is case-sensitive. Backend case-flip ships a silent dead 'Add Coins' button — user thinks the app is broken, not their account.

WAL-B-03 [P1] Tapping Pay before payment-type API completes does not crash or no-op silently

Objective

On cold launch, the login API call that resolves the gateway takes time. If the user taps Pay during that gap, the app must either disable the button or show a 'please wait'.

Steps

1. Force-stop the app. Throttle network to 'Slow 3G' in dev settings (built-in Android dev option, no proxy needed).
2. Cold-launch wallet.
3. Within 1 second of the screen appearing, tap 'Add Coins'.

Expected

Either Pay is disabled until gateway resolves, OR a clear 'preparing payment' indicator is shown. No silent no-op.

Why humans miss it

Users on poor networks always experience this gap. Silent no-ops produce 'app doesn't work' Play Store reviews.

Section C — PhonePe Path

Custom-fetched orderId from your backend, then PhonePeKt SDK checkout, then a server-side status reconciliation. Most production wallet bugs live here.

WAL-C-01 [P0] PhonePe — phone has no UPI app installed	
Objective	App checks for any UPI app via Intent.ACTION_VIEW uri://pay. Verify the user-visible behavior when no UPI app is installed.
Preconditions	Disable / uninstall PhonePe, GPay, Paytm, BHIM. Keep the Cashfree-Hima account intact.
Steps	1. Open wallet. Select a plan. 2. Tap 'Add Coins'.
Expected	User sees a clear message 'No UPI app installed'. Wallet screen returns to a usable state. No coin balance change.
Why humans miss it	Travelers / new-device users routinely have no UPI app on day one. Tested only on dev's daily-driver phones.
WAL-C-02 [P0] PhonePe — user completes payment in PhonePe, kills app before returning	
Objective	Money leaves the user's bank but the activity-result callback is never delivered. On next wallet open, the status must be reconciled and either coins are credited or a clear 'pending' state is shown.
Steps	1. Open wallet, select plan, choose PhonePe path. 2. Complete the UPI PIN in the PhonePe screen so the payment actually debits. 3. Immediately swipe the Hima app out of recents BEFORE the return navigation. 4. Wait 60 seconds. 5. Re-open Hima → Wallet.
Expected	Within ~30 s the wallet reconciles: either coins are credited with a toast, or a clearly-labeled 'pending recharge' card appears with a retry. The recharge is NEVER silently lost.
Why humans miss it	Order-id reconciliation lives only in an in-memory field. Process death between debit and return is the single highest-impact wallet bug class in production.
Branches	(a) Kill while still in PhonePe; (b) kill after PhonePe shows success but before Hima foregrounds; (c) reboot the device between steps 3 and 5.

WAL-C-03 [P0] PhonePe — pay, return showing FAILED, but funds debited (bank lag)

Objective

PhonePe sometimes returns a 'failure' resultCode while the actual UPI debit goes through asynchronously. Confirm no double-spend if the user retries.

Steps

1. Pay via PhonePe. If you can reproduce the 'pending' UPI bank lag, do so by switching to airplane mode between PIN entry and return. Otherwise, simulate by closing network during the return.
2. Wallet shows 'Payment Failed' toast.
3. Tap 'Add Coins' again to retry. Pay successfully this time.
4. Check bank statement / SMS after 5 min for both attempts.

Expected

Either: (a) only the second attempt debits and credits, or (b) both attempts debit AND both credit coins. There is NO case where the user is debited twice and credited once.

Why humans miss it

UPI is asynchronous. Many gateway integrations trust the SDK resultCode and lose the lag-success debit.

WAL-C-04 [P1] PhonePe — back press during PIN screen returns to wallet with usable UI

Steps

1. Open wallet, choose PhonePe path, reach the PhonePe SDK PIN entry screen.
2. Press the system back button repeatedly until back in Hima.
3. Try selecting a plan and paying again immediately.

Expected

Wallet is fully usable; no stuck spinner; Pay button works first tap; no ghost in-progress order indicator.

Why humans miss it

SDK launchers leave the parent activity in 'awaiting result' state. Back-press recovery is rarely tested.

WAL-C-05 [P2] PhonePe — toast on 'PhonePe SDK init failed' is non-blocking and does not leak the merchant id

Objective

If init fails, the user sees a generic toast. Verify it does not include the merchant id, the user id, or any backend URL.

Steps

1. Open wallet under conditions that may make init fail (offline, battery saver, very low memory).
2. Read the toast text exactly. Take a screenshot.

Expected

Toast is generic ('Could not start payment' or similar). No ids visible.

Why humans miss it

Toasts can be screen-shared and OCR'd by accessibility services. Even the merchant id is a fingerprint a competitor would value.

Section D — Cashfree Path

UPI Intent checkout that lets the user pick GPay / PhonePe / BHIM. Reconcile happens on every onResume.

WAL-D-01 [P0] Cashfree — only GPay installed (no PhonePe), the offered app list still lets the user pay

Objective

Cashfree integration offers GPay, PhonePe, BHIM as UPI Intent apps. Confirm graceful behavior when only a subset is installed.

Steps

1. Uninstall PhonePe and BHIM. Keep GPay.
2. Recharge via Cashfree.

Expected

GPay tile is usable; absent apps either hidden or marked 'not installed'. No crash. No 'invisible tap target' that does nothing.

Why humans miss it

Hard-coded UPI app ordering breaks for users who only have one app installed.

WAL-D-02 [P0] Cashfree — order status reconcile after app foreground

Objective

The 'cashfreeLastOrderId' is checked in onResume. Confirm a user who completes payment and is brought back to a foregrounded wallet sees their coins credited or a clear failure.

Steps

1. Pay via Cashfree.
2. Return to wallet. Wait until toast 'Payment Successful' appears AND coin counter updates.
3. Note the exact coin count.
4. Press home, wait 2 min, return to wallet.
5. Compare coin counter on second open.

Expected

Coin counter shown on the second open matches the post-credit count, NOT the pre-recharge count. No duplicate credit if the reconcile runs again.

Why humans miss it

Re-running checkCashfreeOrderStatus on every onResume could credit twice if the local guard variable isn't honored — covered by the 'reset so it won't run again' line but worth verifying with a real round-trip.

WAL-D-03 [P1] Cashfree — user pays, then app receives an incoming call

Steps

1. Pay via Cashfree.
2. While the UPI app is asking for PIN, have a friend call you (real call, not silent).
3. Accept the call. Talk for 30 s.
4. Hang up. Open Hima.

Expected

Wallet either shows the recharge as successful (if the PIN was submitted) with credited coins, or as failed/cancelled. The wallet UI is never frozen with an indefinite spinner.

Why humans miss it

Incoming calls pause the activity in a way unit tests cannot model. Production users hit this constantly.

Section E — Razorpay Path

Backend returns a short_url; the app opens it via a Custom Tab (LauncherActivity). There is no SDK callback — reconciliation is implicit via wallet re-fetch.

WAL-E-01 [P0] Razorpay — short_url opens in Trusted Web Activity, NOT an arbitrary browser

Objective

If the Razorpay link is ever opened in a vanilla browser or a third-party WebView, cookies / form autofill from other sites leak. Confirm the launch uses the in-app Custom Tab.

Steps

1. Tap 'Add Coins' on a Razorpay-assigned account.
2. Watch the transition — note whether the URL bar shows the app icon / lock, or whether it opens Chrome / Samsung Internet / a browser chooser.

Expected

Opens via the LauncherActivity (Custom Tab). Never shows a browser chooser ('Open with...'). Never opens system browser with full URL bar.

Why humans miss it

Intent.ACTION_VIEW on a Razorpay short URL can be hijacked by any app declaring the same intent filter. A user with a malicious browser-impersonator gets phished.

WAL-E-02 [P1] Razorpay — closing the payment tab and returning to wallet shows correct balance

Steps

1. Initiate Razorpay payment.
2. On the payment page, do nothing. Close the Custom Tab via system back.
3. Reopen wallet.

Expected

No coins credited. No 'pending' card. Balance unchanged. Pay button works on retry.

Why humans miss it

Razorpay does not return a result to the host app; the host must self-reconcile on resume. Often forgotten.

WAL-E-03 [P1] Razorpay — backend returns empty short_url

Objective

If the createRazorpay API succeeds but the short_url field is empty/null, the toast is 'Failed to get payment link'. Confirm no half-flow.

Steps

1. Trigger conditions that produce an empty link if available, or repeatedly retry over flaky network and watch for the toast.

Expected

Toast shown. No browser/tab opens. User can retry immediately.

Section F — Google Play Billing (IAP)

BillingManager flow with consumable SKUs. Refund and replay concerns dominate.

WAL-F-01 [P0] GPay IAP — user makes a Play purchase, then issues a Play Store refund within 48 h

Objective

Verify the app handles the refund webhook / does not retain coins after a successful Play Console refund.

Preconditions

Test account that is in Google Play 'license tester' list.

Steps

1. Recharge via GPay IAP. Confirm coins credited.
2. Go to play.google.com → Order history. Request a refund on the order.
3. Wait for refund confirmation email (usually 1–4 hours).
4. Open Hima wallet. Note coin balance.
5. Try to spend the coins on an in-app feature.

Expected

Within a reasonable window the refunded coins are revoked or the account is flagged. At minimum, the user does not retain spendable coins corresponding to a refunded purchase.

Why humans miss it

Refund-abuse is the single largest revenue leak in IAP-based apps. QA almost never tests the refund webhook end-to-end.

WAL-F-02 [P0] GPay IAP — process death between Play purchase success and in-app credit

Objective

After Play's success screen, the app must consume + credit. If killed between the Play callback and the addCoins API, credit must still happen on next launch.

Steps

1. Buy via Play. Complete biometric / PIN. See Play's 'Success' screen.
2. IMMEDIATELY force-stop Hima from Settings → Apps before returning to it.
3. Wait 60 s. Open Hima → Wallet.

Expected

Coins are credited on next launch (Play's pending-purchase delivery mechanism). User sees toast or balance update.

Why humans miss it

Without onResume purchase re-query, the consume() is never called and the user loses both coins and money until Play auto-refunds in 3 days.

Branches

(a) Force-stop; (b) reboot; (c) clear app from recents only (not force-stop).

WAL-F-03 [P0] GPay IAP — rapid double-tap on 'Add Coins' starts only one billing flow

Steps

1. Open wallet. Select plan.
2. Double-tap 'Add Coins' as fast as possible.
3. Triple-tap.
4. Tap repeatedly for 2 s.

Expected

Only one Play purchase sheet appears. Internal trackers (orderId, last_coin_id prefs) are written once, not overwritten with stale values.

Why humans miss it

btnAddCoins uses setOnClickListener (not setOnSingleClickListener as elsewhere in the codebase). Rapid taps can start multiple flows or overwrite SharedPreferences mid-purchase.

WAL-F-04 [P1] GPay IAP — Play Store sign-in expired / no Play account

Steps

1. On a stock device, sign out of all Google accounts in Play Store.
2. Open wallet. Try GPay path.

Expected

Clear user-facing message about Play sign-in. Never a silent no-op or stuck spinner.

WAL-F-05 [P1] GPay IAP — previously purchased and consumed SKU is re-purchasable

Objective

Coin SKUs are consumable (not entitlements). Confirm a user who bought 'coin_14' last week can buy it again.

Steps

1. Buy coin SKU X. Verify credit.
2. Wait 5 min.
3. Buy SKU X again.

Expected

Second purchase succeeds; coins credited again; no 'already purchased' error.

WAL-F-06 [P2] GPay IAP — first ever recharge by a brand-new account fires 'first purchase' analytics exactly once across re-installs

Objective

The 'first_purchase_logged_' SharedPreferences key prevents double-firing the new_user_first_purchase analytics. Confirm behavior across app reinstall.

Steps

1. Sign up a brand-new account.
2. Recharge once. Observe (in Firebase DebugView if available, else trust the toast/log) that new_user_first_purchase fires.
3. Uninstall the app. Reinstall. Log into the same account.
4. Recharge a second time.

Expected

On the second recharge, new_user_first_purchase MUST NOT fire again for the same user id. Marketing attribution stays honest.

Why humans miss it

SharedPreferences is wiped on uninstall, so the local 'already logged' guard is lost. The check should be backend-side. This is a quiet metrics-pollution bug that inflates 'new user conversion' KPIs.

Section G — UPI Gateway Path

Adds a 2% convenience fee on the client side, then opens a payment URL via Intent.ACTION_VIEW.

WAL-G-01 [P0] UPI Gateway — 2% convenience fee is disclosed before the user reaches the gateway

Objective

On UPI Gateway path, total_amount = price * 1.02 (rounded). Verify this is shown on the wallet screen before the user taps Pay, not only inside the UPI app.

Steps

1. Set device to UPI Gateway user account.
2. Open wallet. Note the plan card amount and any near-by fine print.
3. Tap 'Add Coins'.
4. Compare the amount in the UPI app payment screen with the wallet card amount.

Expected

Wallet either shows '+2% convenience fee' near the price, OR the gateway charges exactly the plan price. Silent overcharge is a regulatory issue.

Why humans miss it

Only the UPI Gateway path adds a fee. Users on other gateways see no fee. Same in-app card, different real cost — confusing and abusable.

WAL-G-02 [P1] UPI Gateway — 2% fee math handles ■1 plan, ■9 plan, ■999 plan

Steps

1. Force the smallest plan in the list.
2. Verify rupee value at UPI app: price * 1.02 rounded to integer.
3. Repeat for largest plan.

Expected

Math is consistent: never undercharges, never charges the user more than 2.5% over plan price.

Why humans miss it

Math.round on small amounts can produce ■0 fee. Math.round on ■49.50 produces inconsistent total_amount strings like '50.490000000000005' — UPI rejects malformed amounts on some PSPs.

WAL-G-03 [P0] UPI Gateway — payment URL opens the UPI app, not a browser

Steps

1. Trigger UPI Gateway recharge.
2. Observe what app handles the URL.

Expected

A UPI app (or app chooser limited to UPI apps) opens. NEVER a generic web browser displaying a payment page on http(s).

Why humans miss it

Intent.ACTION_VIEW on a returned payment_url is opaque. If the backend ever returns an https link instead of upi://, the user is silently in a browser with full URL bar visible.

Section H — Privacy & PII Leakage

Privacy bugs that don't require any developer tooling to demonstrate — just a screenshot, the recents tile, TalkBack, or a screen recording.

WAL-H-01 [P0] Recharge amount and coin balance are not visible in the Recent Apps snapshot

Objective

When the user backgrounds Hima during a payment, Android caches a snapshot of the last activity for the Recents view. Confirm the wallet screen blurs / blanks sensitive numbers (coin balance, plan price) in that snapshot.

Steps

1. Set device wallpaper to a high-contrast solid color.
2. Open wallet. Note coin balance and selected plan.
3. Press the system Recent Apps button.
4. Inspect the Hima snapshot tile carefully.
5. Repeat right after tapping 'Add Coins' (gateway sheet visible).

Expected

Either FLAG_SECURE is set on WalletActivity, OR the balance and price text are not legible in the snapshot.

Why humans miss it

A shared phone or one used in a public space leaks financial data on every press of Recents.

WAL-H-02 [P0] Screen-record / screen-share captures the wallet but does not silently include OTP-style data

Steps

1. Start a system screen recording (Android 11+).
2. Open wallet and complete a recharge via PhonePe.
3. Stop recording. Watch playback.

Expected

No OTP or one-time codes are present in the recording from the Hima side. Coin balance and price are visible (acceptable — this is the user's own recording), but no token, no session id, no full backend URL is rendered on screen.

Why humans miss it

Users often screen-record proof of payment to send to support. If a debug Toast or in-screen log shows a token, the user themselves leaks it.

WAL-H-03 [P0] TalkBack does not read out the full coin balance / plan price out loud at full volume in public

Objective

Accessibility services announce focused text. Confirm sensitive amounts are announced with reasonable phrasing, not exposed raw via content descriptions that include the user id or transaction id.

Steps

1. Enable TalkBack.
2. Navigate the wallet end-to-end.
3. Listen for any announcement containing a numeric id, an order id, a coin id, or a backend URL.

Expected

Announcements contain user-facing amounts only. No internal ids.

Why humans miss it

QA usually disables TalkBack. Accessibility users get the full content description, which often includes debug strings.

WAL-H-04 [P1] Coin balance updates only after recharge succeeds — never shows a stale higher balance from a previous session

Objective

On wallet open, the cached balance from SharedPreferences is shown immediately, then refreshed from the API. Confirm the cached value is the user's own, not a previous user on the same device after re-login.

Steps

1. User A logs in, opens wallet, notes balance.
2. Logs out. User B (different account) logs in on same device.
3. User B opens wallet immediately.

Expected

User B sees their OWN balance (initially loading, then fetched), NEVER User A's balance even for one frame.

Why humans miss it

Cached UserData in SharedPrefs across logout is a common session-reuse defect. A balance flash is also a privacy leak — User B sees how much User A had.

WAL-H-05 [P1] Notifications from payment apps during a recharge do not show full UPI ID / bank details on the lock screen

Objective

Hima itself does not control payment-app notifications, but verify any Hima notification fired during recharge (e.g., 'incoming call coming') does not include payment-related context that should not be on a lock screen.

Steps

1. Lock screen with 'Hide sensitive content' enabled.
2. During an active recharge, simulate or wait for an incoming call / chat notification.

Expected

Hima's notifications respect the lock-screen privacy setting. No payment details leak.

WAL-H-06 [P1] Wallet does not write the user's mobile number, email, or full name into the clipboard

Steps

1. Open wallet. Tap and long-press random text labels (especially the coin balance and any 'Order ID' / 'Reference' field on post-purchase screens).
2. Open any notes app and paste.

Expected

Either nothing is copyable, or only the order id (a non-PII value) is copied. Never the user's mobile/email.

WAL-H-07 [P2] External keyboards (SwiftKey, GBoard learn-from-input) do not learn amounts as personalized vocabulary

Objective

Amount entry fields (if any — coupon entry counts) must be marked with IME no-personalized-learning where appropriate.

Steps

1. Enable GBoard personalized learning.
2. Enter a coupon code in the coupon entry field.
3. In any other text input later, type the first 2 characters of the coupon. Check suggestions.

Expected

Coupon codes do not appear in keyboard suggestions elsewhere.

Section I — Process Lifecycle, Backgrounding, Resume

Android can kill your activity at any time. Verify the wallet survives every kill point on every gateway.

WAL-I-01 [P0] Wallet survives configuration change (rotation, fold/unfold) without re-firing a purchase

Steps

1. Open wallet on a foldable / tablet, or enable forced rotation in developer options.
2. Select a plan.
3. Rotate the device twice.
4. Tap 'Add Coins'.

Expected

Selected plan persists. No duplicate API call to addCoins. No double gateway-sheet.

WAL-I-02 [P0] App killed by OS during background while gateway sheet open — recovery is clean

Steps

1. Open wallet. Select a plan. Tap 'Add Coins'. Gateway sheet opens.
2. Background Hima. Open and use ~5 memory-heavy apps to force Hima out of memory.
3. Return to Hima via the Recents tile.

Expected

Cold-start lands on a usable screen (Wallet or Home). No phantom 'order in progress' indicator. No silently-fired duplicate purchase.

WAL-I-03 [P1] Deep-link entry to wallet (from_deeplink = true) — back press goes to MainActivity, not to a stale stack

Steps

1. Tap a deep link to wallet from outside the app (e.g., a notification or a chat message).
2. Wallet opens.
3. Press system back.

Expected

Lands on MainActivity (home), not on a partially-loaded screen, and never on the system launcher.

WAL-I-04 [P1] messageCameWhenIsAlive=1 path — back press behavior when a chat-pop is queued

Steps

1. Trigger a chat / call notification while wallet is open (have another user send a message).
2. Press back from wallet.

Expected

Wallet closes; the pending chat/call surfaces normally. No crash; no double-finish.

WAL-I-05 [P1] Wallet handles 'don't keep activities' developer option

Steps

1. Enable 'Don't keep activities' in Developer options.
2. Open wallet, select a plan, tap 'Add Coins', go through to the gateway, complete payment, return.

Expected

Coins credited; UI consistent; no crash on the return path.

Why humans miss it

This option simulates aggressive OS lifecycle. It exposes Observer leaks and savedInstanceState gaps that production Android sometimes triggers.

Section J — Network Conditions

Airplane mode, hand-offs, captive portals, VPNs.

WAL-J-01 [P0] Plans don't load — user is not stranded on an empty grid with a usable 'Add Coins' button

Steps

1. Enable airplane mode.
2. Open wallet.
3. Observe the screen.
4. Try to tap 'Add Coins'.

Expected

Either a clear 'No connection — retry' state, or the 'Add Coins' button is hidden / disabled. Never a tappable button with no underlying SKU.

WAL-J-02 [P0] Airplane mode flipped on during the gateway sheet

Steps

1. Start a PhonePe / Cashfree recharge.
2. After entering the UPI app, swipe down and enable airplane mode before submitting PIN.
3. Submit PIN.
4. Wait, then disable airplane mode.

Expected

Either the payment fails cleanly and the user sees a clear failure on return, or it eventually settles and reconciles. The user never sees 'success' in the UPI app while Hima shows no balance change permanently.

WAL-J-03 [P1] Wi-Fi to mobile data handover mid-recharge

Steps

1. Connect to Wi-Fi. Open wallet.
2. Start a recharge.
3. While the gateway sheet is open, walk out of Wi-Fi range / disable Wi-Fi.
4. Complete the recharge on mobile data.

Expected

Recharge completes; coins credited; no duplicate.

WAL-J-04 [P1] Captive portal / hotel Wi-Fi blocks the gateway URL

Steps

1. Connect to a captive portal Wi-Fi without logging in.
2. Start a recharge.

Expected

Reasonable error message. Wallet recovers gracefully when the user switches to mobile data and retries.

WAL-J-05 [P2] VPN turned on during recharge (no proxy, just a VPN app)

Steps

1. Enable an off-the-shelf VPN (e.g., Cloudflare WARP).
2. Recharge.

Expected

Works normally. No 'security blocked' from the gateway. (If the app does network-region checks, surface them clearly.)

Why humans miss it

VPN users are routinely flagged as fraud by payment processors. Verify Hima's policy and that the error is user-readable.

Section K — Concurrency & Multi-Device

Same account on two devices. Logout mid-flow. Calls during recharge.

WAL-K-01 [P0] Two devices logged into the same account both recharge at the same time

Objective

Confirm a user does not credit more or less coins than expected when two devices fire near-simultaneous purchases.

Preconditions

Same account on two phones (allowed in this app).

Steps

1. Open wallet on both phones.
2. Coordinate to tap 'Add Coins' within 2 s of each other.
3. Complete both payments.
4. After 2 min, check the final balance on both devices.

Expected

Both purchases credit additively. No purchase lost. No race where one purchase overwrites the other's balance update.

Why humans miss it

Backend has the source of truth, but the app's local afterAddCoins cache can mask one of the two credits visually, leading to a panicked user.

WAL-K-02 [P1] Logout while a recharge is in flight

Steps

1. Start a recharge.
2. On the gateway sheet, leave the payment hanging.
3. On a second device or via a forced-logout admin action, log the account out.
4. Complete the recharge in the gateway.

Expected

App handles the credit gracefully on next login (coins are associated with the original user id). No crash on return to an unauthenticated state.

WAL-K-03 [P1] User is mid-call (audio/video) and recharge dialog appears (deep link / low-balance prompt)

Steps

1. Initiate a call.
2. Trigger a low-balance recharge dialog (let coins run out mid-call).
3. Tap to recharge from inside the call flow.

Expected

Call continues / pauses cleanly. After recharge, the user returns to the call without re-initiating. Coins are deducted only for actual call duration.

Why humans miss it

Recharge from inside an active call is the highest-emotional-stakes flow in the app. Bugs here generate refund tickets.

Section L — Promo, Coupon & Referral Offer Dialogs

The referral dialog mutates the same purchase fields the main Pay button reads. Order of taps matters.

WAL-L-01 [P0] Referral offer dialog can only credit one offer per eligible user, even after app reinstall

Steps

1. Brand-new account. Open wallet. Note referral offer dialog appears (if eligible).
2. Buy via the offer.
3. Uninstall + reinstall Hima. Log into the same account.
4. Open wallet again.

Expected

The offer dialog does NOT appear a second time for an account that already redeemed. The decision is backend-driven, not based on SharedPreferences.

Why humans miss it

Server-side rule is the only safe place to gate one-shot offers. SharedPreferences-only checks lose on uninstall.

WAL-L-02 [P1] Maybe-later on referral dialog — offer is recoverable, not lost

Steps

1. Eligible account opens wallet. Dialog appears.
2. Tap 'Maybe Later'.
3. Reopen wallet.

Expected

Dialog reappears (until purchased or expired). No 'lost forever' state after one dismissal.

WAL-L-03 [P1] Referral dialog Buy Now uses the offer's coin_id, NOT the previously-tapped plan card

Objective

When the referral dialog overrides pointsId/amount/selectedCoin, verify the purchased plan is the offer one — not the plan the user happened to have selected before the dialog appeared.

Steps

1. Open wallet. Manually tap plan B.
2. Trigger the referral offer dialog to appear (relaunch wallet if needed).
3. Tap 'Buy Now' on the dialog.
4. Note the price / coin count in the gateway sheet.

Expected

Gateway charges the OFFER price for the OFFER coin pack, not plan B's price or coins.

Why humans miss it

The dialog mutates the same pointsId / amount fields the main Pay button reads. If the order of mutation flips, a user pays for the offer but receives the wrong plan's coins.

WAL-L-04 [P1] Maybe-later → Buy Now from main button does not accidentally use the offer's pointsId

Steps

1. Referral dialog appears.
2. Tap 'Maybe Later'.
3. Manually tap plan C on the wallet.
4. Tap 'Add Coins'.

Expected

Charges plan C's price, not the offer price.

Why humans miss it

If the dialog 'Maybe Later' code path leaves stale pointsId, the next purchase silently overrides the user's selection.

WAL-L-05 [P2] Referral dialog respects back press as 'Maybe Later'

Steps

1. Dialog appears.
2. Press system back.

Expected

Dialog dismisses cleanly. No purchase fired. No state change.

Section M — Edge Values & Localization

Floating-point math on the 2% fee path. Locale-formatted numbers. Very small and very large amounts.

WAL-M-01 [P0] Plan price formatted to exactly 2 decimal places in the UPI amount field**Objective**

(amountValue + roundedAmount).toString() can produce floating noise like '50.999999999999'. Verify the actual amount the UPI app shows is a clean rupee value.

Steps

1. If accessible, use a plan with price ₹49.50 or ₹99.50 on the UPI Gateway path.
2. Read the rupee amount in the UPI app prompt.

Expected

Two decimal places maximum, no '.99999...' or '.0000001' suffixes. UPI apps reject some malformed amount strings.

WAL-M-02 [P1] Coin counter handles balance going from 9,999,999 to 10,000,000 without UI clipping**Steps**

1. Use an account near 10M coins (test bench) or simulate with many successful recharges.
2. Recharge a small pack.
3. Inspect the coin counter label.

Expected

All digits visible; no ellipsis; localized number formatting.

WAL-M-03 [P1] User who has never bought sees correct '0 coins' state — not empty string, not 'null', not '0.0'**Steps**

1. Fresh account. Wallet open.

Expected

Balance reads '0' (or localized '0'). Never 'null', '0.0', or blank.

WAL-M-04 [P2] Locale changes (English → Hindi → Telugu) do not change the rupee amount sent to gateway**Steps**

1. Recharge on English locale. Note amount.
2. Switch device to Hindi. Recharge.
3. Switch to Telugu. Recharge.

Expected

Same amount across locales. No comma/decimal locale conversion issue (₹1,000 in Hindi 1k separator vs ₹1.000 in European locale).

WAL-M-05 [P2] RTL pseudo-locale doesn't break the coin grid**Steps**

1. Enable 'Force RTL layout direction' in Developer options.
2. Open wallet.

Expected

Grid still selectable; price text not mirrored to nonsense; currency symbol on correct side.

Section N — Multi-Account / Session Boundary

Same device, different accounts. Clear-data. Phone-number change.

WAL-N-01 [P0] Logout immediately after a successful recharge — coins stay credited to the correct account

Steps

1. User A recharges and confirms coin update.
2. Logout within 5 s of credit.
3. User B logs in.
4. User A logs back in.
5. Confirm User A's balance.

Expected

Coins still credited to User A. Account boundary respected.

WAL-N-02 [P0] Clearing app data after a successful recharge does not lose coins

Steps

1. Recharge. Confirm credit.
2. Settings → Apps → Hima → Clear data.
3. Reopen, log in.

Expected

Balance reflects the recharge. (SharedPreferences cache is wiped but the server has the source of truth.)

WAL-N-03 [P1] Phone number change / re-OTP — old recharges are tied to user id, not phone number

Objective

If the app permits phone-number change with re-OTP, verify that the user's coin balance is tied to their stable identifier and survives the change.

Steps

1. On an account with a non-zero balance, change the registered phone number to a different one (if the feature exists).
2. Re-login with new number.

Expected

Balance preserved.

Section O — Refund & Chargeback

Post-credit reversal handling.

WAL-O-01 [P0] User-initiated Razorpay refund — in-app coin balance decreases to match

Objective

If a Razorpay payment is refunded post-credit, the app must either remove the corresponding coins or surface the discrepancy.

Steps

1. Recharge via Razorpay. Confirm credit.
2. Contact support / use the Razorpay dashboard to issue a refund for that order.
3. After refund email, open wallet.

Expected

Within a reasonable window, coins reverse OR the user sees a clear message ('Refund processed — X coins deducted').

WAL-O-02 [P1] Chargeback dispute scenario — repeated recharge on a flagged card surfaces an error

Steps

1. Use a card that has had recent chargebacks (per the gateway's merchant rules). Attempt to recharge.

Expected

Gateway-side refusal is surfaced clearly. No repeated retry loop draining the user's card.

Section P — Dynamic Server Content (Banner)

Banner_image is fully backend-controlled. Verify it can't crash, freeze, or redirect.

WAL-P-01 [P1] Banner image loaded from backend is constrained in size — a 50MB image doesn't crash the wallet

Steps

1. Coordinate to point the banner_image URL at a large file (50MB+) in a test environment.
2. Open wallet.

Expected

Wallet either loads with a placeholder or gracefully falls back. No OOM crash. No multi-second freeze.

WAL-P-02 [P2] Broken banner_image URL — wallet still usable

Steps

1. Test environment returns banner_image = a 404 URL.
2. Open wallet.

Expected

Banner area is empty / placeholder. Plans still load and are purchasable.

Section Q — Deep Links to Wallet

Wallet can be opened from notifications, chat messages, and external URLs. The intent surface must be hardened.

WAL-Q-01 [P0] Deep link to wallet from a chat message respects the intent's source — coins are credited to the receiver, not the sender

Steps

1. User A sends a wallet deep link to User B via in-app chat.
2. User B taps the link.

Expected

Wallet opens for User B (their balance, their account). No User A context.

WAL-Q-02 [P1] Pasting a long unknown deep-link URL with strange query params doesn't crash the wallet

Steps

1. Open the device browser. Paste a deep link variation with extra unknown params (e.g., ?coin_id=99999&debug;=1&auto;_buy=true).
2. Open the link. Confirm what happens.

Expected

Wallet opens normally OR refuses with a clear message. NEVER auto-initiates a purchase based on a URL param.

Why humans miss it

If 'auto_buy' style params ever exist, an attacker can send such a link via WhatsApp / SMS and the user opens straight to a charged purchase.

WAL-Q-03 [P2] App-link sharing — Wallet has no sensitive query params in the share URL

Steps

1. If wallet has a share function, share to clipboard.

Expected

Shared URL does not contain user id, mobile, email, or auth token.

Branching Scenario Trees

The cases above are atomic. Real users hit them in combinations. These trees describe full journeys with every branch a QA should verify. Each branch references atomic cases above where applicable.

Scenario B-1 — PhonePe: 'I paid but no coins' user journey

User opens wallet, selects plan, taps Add Coins	→ B-1.1
B-1.1 PhonePe SDK init failed (offline / battery saver)	→ generic toast. User retries. Sub-branches:
B-1.1.a Network restored before retry	Expected: retry succeeds normally.
B-1.1.b User still offline	Expected: same toast. No partial state.
B-1.2 SDK init OK → fetchOrderFromBackend	Sub-branches on backend response:
B-1.2.a token/orderId returned	Continue to PhonePe checkout.
B-1.2.b backend 5xx / timeout	Toast 'API Failure'. User retries — must not create a duplicate order id on success.
B-1.2.c backend returns 200 but malformed JSON	Toast 'Invalid server response'. No phantom orderId stored locally.
B-1.3 PhonePe checkout UI opens	Sub-branches:
B-1.3.a User completes UPI PIN, returns to app, checkOrderStatus → COMPLETED	Coins credited. Analytics fire. Balance refreshes.
B-1.3.b User completes UPI PIN, app killed before return	(WAL-C-02) — On reopen, reconcile must run. Expected: credit or clear pending state.
B-1.3.c User cancels in PhonePe	resultCode = cancelled. Wallet shows no toast OR clear 'cancelled' message. No charge.
B-1.3.d Network drop during PIN entry	PhonePe shows pending/failure. checkOrderStatus on return should reflect actual bank state.

Scenario B-2 — Google Play IAP: refund-abuse defense

User buys SKU via Play Billing	Coins credited via addCoinsGpay.
B-2.1 User immediately requests Play refund (within 2 h)	Sub-branches:
B-2.1.a User has not yet spent the coins	Expected: server-side webhook reverses balance to pre-purchase level. App reflects on next open.
B-2.1.b User has spent half the coins on a call	Expected: server policy — either negative balance allowed (account flagged) or refund denied. Either way, no infinite coin exploit.
B-2.1.c User has spent all coins	Expected: account flagged. Future recharges may require review.
B-2.2 User requests Play refund AFTER 48h	Different Play policy. Server should still receive a webhook and reconcile.
B-2.3 Refund processed during process death	On next launch, app must not 're-credit' the original purchase by re-running pending-purchase processing.

Scenario B-3 — Process death during payment, every path

User reaches gateway sheet, then app is killed	Branch on gateway:
B-3.PhonePe	On reopen, lastOrderId is lost (in-memory). checkOrderStatus never runs for that order. Expected: server-side webhook independently credits coins; app sees updated balance on profile refresh.

B-3.Cashfree	On reopen, cashfreeLastOrderId is lost. Same expectation: server-side reconciliation.
B-3.Razorpay	No SDK callback; user returns via system back. App must check transaction history / latest balance on wallet open.
B-3.GPay	Pending-purchase delivery via Play Billing on next launch is the recovery path. Expected: BillingManager queries on init and processes deferred purchases.
B-3.UPI Gateway	Payment URL was already opened in UPI app. App must re-fetch balance and surface 'recent recharge in your bank' help text.
ALL branches must converge	User is never simultaneously: (a) debited by bank, (b) without coins, (c) without a clear in-app message.

Scenario B-4 — Coupon / Referral dialog interaction matrix

Wallet opens with both: a manually-tapped plan AND a referral dialog appearing	Order matters:
B-4.a User tapped plan B, dialog appears, user taps 'Buy Now'	Expected: charges OFFER, not plan B.
B-4.b Dialog appears first, user taps Maybe Later, then taps plan B → Add Coins	Expected: charges plan B, NOT offer (no stale pointsId).
B-4.c Dialog appears, user dismisses with back, taps Add Coins on default plan	Expected: charges default (first) plan.
B-4.d User scrolls coin grid while dialog is open	Expected: dialog stays modal; grid scroll under it is either disabled or harmless.
B-4.e Dialog appears twice if API returns show_dialog=true twice (e.g., after backgrounding)	Expected: dialog appears at most once per wallet session, OR backend prevents repeat triggers.

Scenario B-5 — Shared / public-space device usage

Family member uses the phone after a recharge	Privacy checks:
B-5.a Recents tile shows wallet snapshot	(WAL-H-01) Balance must be obscured or FLAG_SECURE.
B-5.b Notification panel shows a Hima notification	Must respect lock-screen sensitivity setting.
B-5.c Clipboard inspected	No PII / no order id / no token.
B-5.d Friend opens app via fingerprint of phone owner (no app-level lock)	Wallet visible. Document as acceptable OR add an in-app lock screen for the wallet section.
B-5.e Phone is screen-mirrored to TV during recharge	Sensitive numbers visible — acceptable IF user knows. Verify no debug strings.

Observable Code-Smell Checks

These items were spotted while reading the production wallet code. Every check below is observable from outside the app — no code change, no root, no proxy. QA reports a failure; engineering decides how to fix.

Concern	How to verify by observation
Toasts and Logcat messages	If you tether your phone to a laptop and turn on USB debugging (no root, just `adb logcat`), you can see what the app prints. The dev team can run this. Verify the following strings DO NOT appear in release builds: 'SelectedCoinID', 'PhonePeResponse', 'PhonePeOrderStatus', 'JWT_TOKEN: Generated Token:', 'paymentGateway \$...', 'CoinResposneData', 'CashfreeOrderResponse', 'CashfreeOrderStatus'. (P0)
Pay button single-tap protection	The 'Add Coins' button uses <code>setOnClickListener</code> , not <code>setOnSingleClickListener</code> — even though <code>setOnSingleClickListener</code> is used elsewhere in the same file (<code>cvBack</code> , <code>llRecharge</code>). Verify a rapid triple-tap doesn't double-start anything. (P0 — WAL-F-03)
PhonePe init logging in production	<code>PhonePeKt.init</code> is called with <code>enableLogging = true</code> . In a release build, enable USB debugging and capture logcat during a real recharge. The SDK should NOT print payment payloads. If it does, ship a build with <code>enableLogging = BuildConfig.DEBUG</code> . (P0)
Banner image source	<code>banner_image</code> URL is loaded by Glide with no allowlist. Have backend team confirm the image is served from a trusted CDN. From the app, just verify a missing image doesn't crash (WAL-P-02). (P1)
Cached coin count flash	<code>binding.tvCoins.text</code> is set to <code>userData?.coins</code> immediately on screen open from <code>SharedPreferences</code> cache, then refreshed via <code>getCoins</code> . Test the user-switch case: WAL-H-04. (P1)
In-memory order id lifetime	<code>lastOrderId</code> and <code>cashfreeLastOrderId</code> are non-persisted class fields. Process death loses them. WAL-C-02, WAL-D-02 directly test this. (P0)
Settings observer fires <code>getSettings</code> on every <code>onResume</code>	The settings API and login API both run on every wallet <code>onResume</code> . Test: open wallet, background and foreground 10x. Check if you see duplicate gateway toasts or any 'Please try again later' surface. (P2)
Crash on malformed PhonePe status response	JSONObject parsing in <code>checkOrderStatus</code> assumes keys exist. Test behavior when the backend is briefly degraded and returns unexpected JSON (you can replicate by retrying during known maintenance windows). (P1)

Execution Checklist

Before signing off a wallet release, complete the matrix below.

Device	OS	Gateway	Network	P0 cases pass
Low-end Android (Redmi/Realme, 3GB RAM)	Android 10	PhonePe	Wi-Fi	■
Low-end Android (Redmi/Realme, 3GB RAM)	Android 10	PhonePe	Mobile 4G	■
Low-end Android (Redmi/Realme, 3GB RAM)	Android 10	PhonePe	Slow 3G	■
Low-end Android (Redmi/Realme, 3GB RAM)	Android 10	PhonePe	Airplane→Recover	■
Low-end Android (Redmi/Realme, 3GB RAM)	Android 10	Cashfree	Wi-Fi	■
Low-end Android (Redmi/Realme, 3GB RAM)	Android 10	Cashfree	Mobile 4G	■
Low-end Android (Redmi/Realme, 3GB RAM)	Android 10	Cashfree	Slow 3G	■
Low-end Android (Redmi/Realme, 3GB RAM)	Android 10	Cashfree	Airplane→Recover	■
Low-end Android (Redmi/Realme, 3GB RAM)	Android 10	Razorpay	Wi-Fi	■
Low-end Android (Redmi/Realme, 3GB RAM)	Android 10	Razorpay	Mobile 4G	■
Low-end Android (Redmi/Realme, 3GB RAM)	Android 10	Razorpay	Slow 3G	■
Low-end Android (Redmi/Realme, 3GB RAM)	Android 10	Razorpay	Airplane→Recover	■
Low-end Android (Redmi/Realme, 3GB RAM)	Android 10	GPay IAP	Wi-Fi	■
Low-end Android (Redmi/Realme, 3GB RAM)	Android 10	GPay IAP	Mobile 4G	■
Low-end Android (Redmi/Realme, 3GB RAM)	Android 10	GPay IAP	Slow 3G	■
Low-end Android (Redmi/Realme, 3GB RAM)	Android 10	GPay IAP	Airplane→Recover	■
Low-end Android (Redmi/Realme, 3GB RAM)	Android 10	UPI Gateway	Wi-Fi	■
Low-end Android (Redmi/Realme, 3GB RAM)	Android 10	UPI Gateway	Mobile 4G	■
Low-end Android (Redmi/Realme, 3GB RAM)	Android 10	UPI Gateway	Slow 3G	■
Low-end Android (Redmi/Realme, 3GB RAM)	Android 10	UPI Gateway	Airplane→Recover	■
Mid-range (Samsung A-series)	Android 13	PhonePe	Wi-Fi	■
Mid-range (Samsung A-series)	Android 13	PhonePe	Mobile 4G	■
Mid-range (Samsung A-series)	Android 13	PhonePe	Slow 3G	■
Mid-range (Samsung A-series)	Android 13	PhonePe	Airplane→Recover	■
Mid-range (Samsung A-series)	Android 13	Cashfree	Wi-Fi	■
Mid-range (Samsung A-series)	Android 13	Cashfree	Mobile 4G	■
Mid-range (Samsung A-series)	Android 13	Cashfree	Slow 3G	■
Mid-range (Samsung A-series)	Android 13	Cashfree	Airplane→Recover	■
Mid-range (Samsung A-series)	Android 13	Razorpay	Wi-Fi	■

Device	OS	Gateway	Network	P0 cases pass
Mid-range (Samsung A-series)	Android 13	Razorpay	Mobile 4G	■
Mid-range (Samsung A-series)	Android 13	Razorpay	Slow 3G	■
Mid-range (Samsung A-series)	Android 13	Razorpay	Airplane→Recover	■
Mid-range (Samsung A-series)	Android 13	GPay IAP	Wi-Fi	■
Mid-range (Samsung A-series)	Android 13	GPay IAP	Mobile 4G	■
Mid-range (Samsung A-series)	Android 13	GPay IAP	Slow 3G	■
Mid-range (Samsung A-series)	Android 13	GPay IAP	Airplane→Recover	■
Mid-range (Samsung A-series)	Android 13	UPI Gateway	Wi-Fi	■
Mid-range (Samsung A-series)	Android 13	UPI Gateway	Mobile 4G	■
Mid-range (Samsung A-series)	Android 13	UPI Gateway	Slow 3G	■
Mid-range (Samsung A-series)	Android 13	UPI Gateway	Airplane→Recover	■
Flagship (Pixel / OnePlus)	Android 14	PhonePe	Wi-Fi	■
Flagship (Pixel / OnePlus)	Android 14	PhonePe	Mobile 4G	■
Flagship (Pixel / OnePlus)	Android 14	PhonePe	Slow 3G	■
Flagship (Pixel / OnePlus)	Android 14	PhonePe	Airplane→Recover	■
Flagship (Pixel / OnePlus)	Android 14	Cashfree	Wi-Fi	■
Flagship (Pixel / OnePlus)	Android 14	Cashfree	Mobile 4G	■
Flagship (Pixel / OnePlus)	Android 14	Cashfree	Slow 3G	■
Flagship (Pixel / OnePlus)	Android 14	Cashfree	Airplane→Recover	■
Flagship (Pixel / OnePlus)	Android 14	Razorpay	Wi-Fi	■
Flagship (Pixel / OnePlus)	Android 14	Razorpay	Mobile 4G	■
Flagship (Pixel / OnePlus)	Android 14	Razorpay	Slow 3G	■
Flagship (Pixel / OnePlus)	Android 14	Razorpay	Airplane→Recover	■
Flagship (Pixel / OnePlus)	Android 14	GPay IAP	Wi-Fi	■
Flagship (Pixel / OnePlus)	Android 14	GPay IAP	Mobile 4G	■
Flagship (Pixel / OnePlus)	Android 14	GPay IAP	Slow 3G	■
Flagship (Pixel / OnePlus)	Android 14	GPay IAP	Airplane→Recover	■
Flagship (Pixel / OnePlus)	Android 14	UPI Gateway	Wi-Fi	■
Flagship (Pixel / OnePlus)	Android 14	UPI Gateway	Mobile 4G	■
Flagship (Pixel / OnePlus)	Android 14	UPI Gateway	Slow 3G	■
Flagship (Pixel / OnePlus)	Android 14	UPI Gateway	Airplane→Recover	■
Tablet (10")	Android 12	PhonePe	Wi-Fi	■
Tablet (10")	Android 12	PhonePe	Mobile 4G	■
Tablet (10")	Android 12	PhonePe	Slow 3G	■
Tablet (10")	Android 12	PhonePe	Airplane→Recover	■
Tablet (10")	Android 12	Cashfree	Wi-Fi	■
Tablet (10")	Android 12	Cashfree	Mobile 4G	■
Tablet (10")	Android 12	Cashfree	Slow 3G	■
Tablet (10")	Android 12	Cashfree	Airplane→Recover	■
Tablet (10")	Android 12	Razorpay	Wi-Fi	■
Tablet (10")	Android 12	Razorpay	Mobile 4G	■
Tablet (10")	Android 12	Razorpay	Slow 3G	■
Tablet (10")	Android 12	Razorpay	Airplane→Recover	■
Tablet (10")	Android 12	GPay IAP	Wi-Fi	■
Tablet (10")	Android 12	GPay IAP	Mobile 4G	■
Tablet (10")	Android 12	GPay IAP	Slow 3G	■
Tablet (10")	Android 12	GPay IAP	Airplane→Recover	■

Device	OS	Gateway	Network	P0 cases pass
Tablet (10")	Android 12	UPI Gateway	Wi-Fi	■
Tablet (10")	Android 12	UPI Gateway	Mobile 4G	■
Tablet (10")	Android 12	UPI Gateway	Slow 3G	■
Tablet (10")	Android 12	UPI Gateway	Airplane→Recover	■

Bug Report Template

When a case fails, file with the structure below. Money / privacy bugs are easy to misreport — this template forces the right detail.

Title	[Gateway]
Test Case ID	WAL-?-??
Priority observed	P0 / P1 / P2
Device + OS	e.g., Redmi Note 11, Android 13
Account state	Coin balance before, plan selected, gateway assigned
Network	Wi-Fi / 4G / Slow 3G / airplane toggled at step X
Reproducibility	1/1, 3/5, etc.
Steps (numbered)	Exact taps, with timestamps if timing matters
Actual behavior	Observable only — no assumptions about internals
Expected behavior	Quote the expected from this document
Money impact	Yes / No / Unclear. If yes, attach bank SMS / Play receipt
Privacy impact	What can a person standing behind the user see / what does a screenshot reveal
Screen recording	Attach mp4. Mandatory for P0.
Logs (if dev provides)	adb logcat snippet — only if engineering can capture, never user-side

End of document. Last updated: 2026-05-11.